

Distributed Control Framework for Connectivity Preservation and Collective Reconfiguration in Flow-Driven Robot Fleets

Prajwal Dutta, Geoff Hollinger, and Xi Yu

Abstract—We study the deployment of underwater robot fleets in large, flow dominated environments where localization is denied, communication is limited, and energy is scarce. The robots need to travel long distances, detect emerging targets, and stay connected using only local information. To meet these challenges we design a distributed framework that exploits natural drift and activates control only when required. The method begins with the robots deployed as a fully connected network. Using minimal information exchange, each robot prunes non-critical edges, producing a sparse and tree like structure that spans the entire team. Upon this structure we construct a distributed control Lyapunov function–control barrier function (CLF–CBF) framework. Each robot enforces barrier constraints to maintain the necessary connections, while redundant links are discarded when alternative paths emerge. When a target is discovered, this framework ensures the detecting robot attends the target, while the team reconfigures to provide a connected formation. This approach guarantees long term connectivity with very low control effort, and enables collective response to dynamic tasks. Simulations validate the framework and demonstrates that it yields spanning structures that converge toward a minimum spanning tree over time, highlighting both its efficiency and robustness under realistic underwater conditions.

I. INTRODUCTION

We are interested in deploying a fleet of underwater robots into large-scale, challenging environments such as the deep ocean and under-ice cavities, where localization is denied, communication bandwidth is limited, and the environment is dominated by dynamic flows and eddies [1]. These robots must traverse long distances on limited energy budgets, discover emerging regions of interest, sample the environment, and relay data, all while relying on intermittent and bandwidth-limited communication [2], [3].

The central challenge is to design strategies that enable the fleet to (i) expend minimal control effort while traveling long distances and completing complex tasks, (ii) efficiently cover a large spatial domain to discover emerging targets while maintaining network connectivity, and (iii) realize a globally connected configuration using only localized information and limited communication.

Our proposed approach is to deploy the robots together and then allow them to drift with the prevailing flow as much as possible. Control effort is only applied when it is necessary.

This includes situations when the robots risk drifting outside a prescribed region or when they risk losing connectivity. In this way the formation disperses while staying connected, and the group can also reconfigure when a new target location is discovered or generated [4]. This strategy is bold, but it is supported by recent progress. Previous work has shown that surface vehicles can drift with the flow while using minimal control to remain in gyre like structures [5], or track the periodicity of revisits via intermittent control triggered by prescribed conditions [6]. In addition, broadly adopted models describe the uncontrolled motion of objects in water as drift combined with Brownian type diffusion [7]–[9], which align with observation on underwater robots [10], [11]. This indicates that our fleet will approximately follow the main direction of the flow while also dispersing stochastically.

Although dispersion promotes coverage of large task spaces, two challenges remain open. The first is how to maintain connectivity. The second is how to enable reconfiguration around targets while working with strict energy budgets and unreliable communication that cannot support centralized control. Prior work on connectivity maintenance and target surrounding has emphasized the importance of graph theoretic methods [12], [13]. Control barrier functions (CBFs) have received attention for their energy efficiency in this setting. For example, CBFs have been applied to the Fiedler value of the network graph in order to maintain connectivity [14]. This approach is effective but it requires centralized and computationally expensive eigenvalue calculations. Other work has applied distributed CBF formulations that keep robots pairwise within communication range [15], [16]. More recently, a hybrid framework was introduced that combines distributed CBFs with a graph theoretic tool, where each robot interacts only with a limited set of neighbors defined by a minimum spanning tree (MST) [17]. This solution is promising since each robot only applies CBF to itself and to selected neighbors, but it still requires generating a minimum spanning tree that depends on global information.

In our case a feasible framework must combine two ingredients. The first is a distributed neighbor selection strategy so that each robot chooses a limited set of neighbors to remain within communication range. The second is a local control law applied only to these neighbors. Together these two features must guarantee global connectivity while minimizing control effort. Work on spanning trees has shown that distributed verification methods exist, but the actual construction of a minimum spanning tree is classically cen-

This work was supported by the NSF Awards 2322055 and 2529791.

P. Dutta and X. Yu is with the School of Manufacturing Systems and Networks at Arizona State University (e-mail: {pdutta9, xyu}@asu.edu). G. Hollinger is with the Department of Mechanical, Industrial, and Manufacturing Engineering at Oregon State University (e-mail: Geoff.Hollinger@oregonstate.edu).

tralized [18]–[20]. Progress in fully distributed protocol for exact MST construction [21], [22] remain communication-intensive. As a result, several approximate or near-optimal distributed methods have been proposed, but they do not achieve exact solutions [23].

We take advantage of two physical features of our problem. First, the robots are deployed together, and the initial configuration forms a fully connected graph. Second, the purpose of a spanning tree is to reduce long range connections. A robot should drop links to distant neighbors whenever a closer neighbor can maintain connectivity at lower control cost. Recent progress [24] has also shown that robots can verify whether an edge is critical, meaning that breaking it would break the network connectivity, using distributed procedures that rely only on very limited data sharing. Based on these observations we propose a distributed method that requires very little information exchange. Each robot shares only a list of its neighbor indices. With this limited data the robots identify and prune edges that are not critical to connectivity. The result is a sparse and tree-like structure that spans the entire network. Once this structure is established, we construct a distributed control Lyapunov function–control barrier function (CLF–CBF) framework upon it. Each robot maintains only the necessary connections by enforcing CBF constraints, and redundant links are pruned whenever new edges provide alternative paths. When a target is detected, the framework drives the robot that discovers it toward the target, while the rest of the team reconfigures to form a supporting network from the bottom up. As a result, the team preserves connectivity, expands to maintain coverage, and collectively shifts toward the target location.

Simulations demonstrate that this method can efficiently attend to emerging targets in different regions of the environment while keeping the network connected. Over deployment time the spanning structure produced by this distributed pruning strategy converges toward a minimum spanning tree. This shows both its long term efficiency and its robustness in flow dominated and resource constrained conditions.

This paper is organized as follows. Sec. II introduces the problem. Sec. III presents our two-fold control framework. Sec. IV demonstrates the results of the proposed method via simulated examples. Sec. V concluded this work.

II. BACKGROUND AND PROBLEM FORMULATION

A. Advection-diffusion model [7]–[9]

Considering a group of N objects released together in the water, their collective motion can be decomposed into a drift component and a stochastic dispersion component. Let $x_i(t) \in \mathbb{R}^n$ denote the position of the i -th object at time t , where $i \in \{1, 2, \dots, N\}$, and let

$$\bar{x}(t) = \frac{1}{N} \sum_{i=1}^N x_i(t)$$

be the center of mass of the group. The evolution of the center of mass is governed primarily by the advective flow field, i.e.,

$$\dot{\bar{x}}(t) \approx f_{\text{flow}}(\bar{x}(t), t),$$

where f_{flow} is the background velocity field. Each individual object then deviates from $\bar{x}(t)$ due to unresolved small-scale turbulence and fluctuations, which can be modeled as Brownian motion:

$$dx_i(t) = f_{\text{flow}}(\bar{x}(t), t)dt + dW_i(t),$$

where $W_i(t)$ is a Wiener process. Thus the group drifts together with the flow through the motion of $\bar{x}(t)$, while individuals spread stochastically around the drifting center due to the independent diffusion terms.

As an example, in the simulation platform where $n = 2$, we adopt an advection field consisting of a uniform background current superimposed with a set of Q vortices:

$$f_{\text{flow}}(x, t) = v_{\text{bg}}(t) + \sum_{q=1}^Q \kappa_q \begin{bmatrix} -(x_2 - x_{2,q}) \\ (x_1 - x_{1,q}) \end{bmatrix},$$

with vortex centers $(x_{1,q}, x_{2,q})$ and strengths κ_q , $q = 1, 2, \dots, Q$. Drag applied on robot i is $F_{\text{drag}}(\dot{x}_i) = -\gamma \dot{x}_i$, $\gamma > 0$. A simulation-accurate model is

$$dx_i(t) = u_i(t)dt + f_{\text{flow}}(\bar{x}(t), t)dt + F_{\text{drag}}(\dot{x}_i)dt + dW_i(t),$$

where u_i is the control. For control synthesis, we absorb drag and stochastic effects into a bounded disturbance d_i to retain control-affine form:

$$\dot{x}_i(t) = f_{\text{flow}}(\bar{x}(t), t) + u_i(t) + d_i(t), \quad \|d_i\| \leq \bar{d}. \quad (1)$$

With the control input $u_i(t)$ set to zero throughout the deployment time window, the robots will be passively advected by the background flow while gradually dispersing due to stochastic perturbations.

Notice that the choice of a two-dimensional environment is made solely for ease of visualization. The developed method is dimension-agnostic and extends directly to three-dimensional settings.

B. Assumptions on communication and connectivity

We model these N robots indexed by $i \in \mathcal{V} = \{1, \dots, N\}$, each with planar position $x_i \in \mathbb{R}^2$ and bounded control input $u_i \in \mathbb{R}^2$, with $\|u_i\| \leq u_{\text{max}}$. Robots follow single-integrator dynamics

$$\dot{x}_i = u_i,$$

and a minimum pairwise separation distance $d_{\text{min}} > 0$ must be maintained to avoid collisions.

The communication network is described by the standard disk model with nominal range $R_{\text{max}} > 0$. At time t , an undirected edge (i, j) exists between robots i and j if $\|x_i(t) - x_j(t)\| \leq R_{\text{max}}$. The resulting network is represented by the undirected graph $\mathcal{G}_t = (\mathcal{V}, \mathcal{E}_t)$, where the vertex set $\mathcal{V}$ corresponds to robots and the edge set $\mathcal{E}_t$ encodes pairwise communication links. The neighborhood of robot i is defined as

$$\mathcal{N}(i) = \{j \in \mathcal{V} : \|x_i(t) - x_j(t)\| \leq R_{\text{max}}\},$$

so that agent i observes relative states and exchanges messages only with robots in $\mathcal{N}(i)$. We denote the degree of node i as $\deg(i) = |\mathcal{N}(i)|$. The set of common neighbors of

two nodes i, j is $\mathcal{N}(i) \cap \mathcal{N}(j)$, and the two-hop neighbors of i are $\mathcal{N}^{(2)}(i) = \{k \in \mathcal{V} : \exists j \in \mathcal{N}(i) \text{ with } k \in \mathcal{N}(j)\}$. The initial communication graph $\mathcal{G}_0 = (\mathcal{V}, \mathcal{E}_0)$ is assumed to be connected. Connectivity is defined in the standard graph-theoretic sense: the graph $\mathcal{G}_t$ is connected if, for every pair $i, j \in \mathcal{V}$, there exists a path in $\mathcal{E}_t$ linking them. No global positions, centralized maps, or spectral information are assumed to be available.

C. Problem Statement

Consider the robots and communication network introduced in the previous section. The robots are advected by the background flow field f_{flow} and subject to stochastic perturbations that induce dispersion (as introduced in eq. (1)). This natural dispersion promotes exploration but risks breaking connectivity. Locations of interest are not known a priori and are only detected when an agent enters its sensing footprint. During exploration phases (when no target is detected), the team drifts with f_{flow} while maintaining connectivity and sparsifying the communication graph. When a target is detected, a local subset of agents initiates goal-seeking behavior while the remaining agents act as relays to preserve end-to-end connectivity across the fleet.

Each agent i has access to: (i) its own state x_i ; (ii) relative states $\{x_j - x_i\}_{j \in \mathcal{N}(i)}$ and messages received from neighbors; and (iii) local graph quantities that can be computed through neighbor exchanges, such as two-hop information. No centralized computation, global identifiers beyond local exchanges, globally shared maps, spanning trees, or spectral quantities (e.g., eigenvalues) are assumed.

The problem is therefore to design a **distributed control framework** that satisfies the following objectives:

- 1) **Connectivity preservation:** ensure that the communication graph $\mathcal{G}_t$ remains connected for all t .
- 2) **Energy efficiency:** leverage flow advection and activate inputs only when necessary.
- 3) **Collective reconfiguration:** enable the team to spread out for coverage during exploration and shift toward newly discovered targets, while maintaining connectivity and collision avoidance.

The solution must rely exclusively on local relative information and distributed exchanges, without requiring any centralized solver or globally shared information.

III. METHODOLOGY

Our method has two components. First, we identify a sparse critical communication structure by pruning non-essential edges. Then, we apply a distributed CLF-CBF framework on this structure to preserve connectivity and enable target-driven reconfiguration. Each component is detailed in the following subsections.

A. Distributed Identification of a critical graph structure

We first identify the set of edges that are *critical*, meaning that removing them would disconnect the network. Following the neighbor-only procedure in [24], each robot exchanges simple messages with its neighbors until it can determine

whether a link provides the only route to part of the network. If so, the link is marked as critical and must never be removed. This produces the set $\mathcal{C} \subseteq \mathcal{E}$ of edges that guarantee global connectivity.

Once $\mathcal{C}$ is identified, our method sparsifies the network by systematically pruning among the remaining edges $\mathcal{E}^- = \mathcal{E} \setminus \mathcal{C}$. This pruning is guided by three key principles: 1) prioritize short alternatives, 2) ensure local safety, and 3) maintain global robustness.

Distance-first pruning: Let $l_{ij} = \|x_i - x_j\|$ denote the length of edge $(i, j) \in \mathcal{E}^-$. We say that a *shorter alternative* between i and j exists if there is a path $i \rightsquigarrow j$ of at most H hops (default $H = 2$) such that

$$\max_{(p,q) \in i \rightsquigarrow j} l_{pq} < l_{ij}.$$

This condition ensures that if the long edge (i, j) is removed, i and j are still connected through a strictly shorter route.

Pruning guards: An edge $(i, j) \in \mathcal{E}^-$ is eligible for removal only if all the following conditions hold:

- 1) **Alternative check:** A shorter alternative $i \rightsquigarrow j$ exists.
- 2) **Degree guard:** Neither endpoint becomes isolated, i.e., $\min\{\deg(i), \deg(j)\} \geq 2$.
- 3) **Next-step guard:** All edges on the alternative path satisfy $l_{pq} \leq R_{\max} - \varepsilon$, for some margin $\varepsilon > 0$, reducing the risk that small drift immediately breaks the substitute path.
- 4) **Critical guard:** $(i, j) \notin \mathcal{C}$.

Only when all four checks pass is (i, j) removed.

Conflict-free batching: To avoid transient disconnection from simultaneous deletions, pruning is executed in batches. A batch $\mathcal{E}_{\text{batch}} \subseteq \mathcal{E}^-$ is constructed such that no two edges in $\mathcal{E}_{\text{batch}}$ share a vertex. All edges in the batch are then pruned atomically:

$$\mathcal{E} \leftarrow \mathcal{E} \setminus \mathcal{E}_{\text{batch}}.$$

A three-step process of pruning towards a critical structure is summarized as in Algorithm 1. Besides it, one can add the following optional operation for a more robust result.

Optional robustness via triangle-cycle preservation: In practice, communication can be intermittent, so we retain redundancy by preserving short local 3-cycles whenever possible. For edge (i, j) , define the common-neighbor count

$$T(i, j) = |\mathcal{N}(i) \cap \mathcal{N}(j)|,$$

and the normalized density score

$$R(i, j) = \frac{T(i, j)}{\min\{\deg(i), \deg(j)\}}.$$

A combined robustness score is then

$$P(i, j) = R(i, j) (1 + T(i, j)), \quad P_{\text{bil}}(i, j) = \frac{1}{2}(P_i + P_j).$$

When two edges have comparable length, the pruning rule keeps the one with the higher P_{bil} , preserving denser triangles that act as two-hop backups.

Numerical Example (see Fig. 1): Here we give a simple example to illustrate the process. The original graph has five

nodes and six edges, including cycles (2, 3, 4) and (3, 4, 5). Using the method of [24], the tail (1, 2) is marked critical while the cycle edges are non-critical.

After Step 1: edge (3, 4) is removed because the path $3 \rightarrow 5 \rightarrow 4$ is shorter and all guards hold.

After Step 2: edge (4, 5) is removed because the path $4 \rightarrow 3 \rightarrow 2 \rightarrow 5$ is available and safe. The graph is reduced to four edges, remains connected, and is tree-like.

Optionally, edge (2, 5) can be added as a two-hop shortcut to improve robustness while keeping the structure sparse.

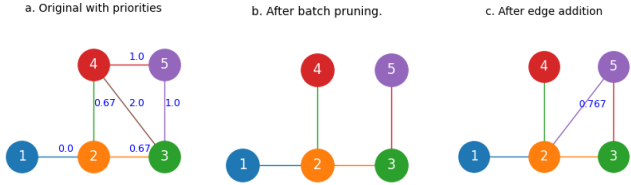


Fig. 1. Edge Priority-Based Graph Modification on a 5-Node Network: (a) Original graph with priorities; (b) after batch pruning of edges (3, 4) and (4, 5); (c) after adding candidate edge (2, 5), annotated with its score $S_{\text{add}} = 0.767$.

Resulting structure: This three-step process yields a sparse, tree-like backbone $\mathcal{G}' = (\mathcal{V}, \mathcal{E}')$. Unlike a global MST, which requires centralized knowledge of all edge lengths, this backbone is constructed using only neighbor-to-neighbor information. The benefits are: (i) reduced the edge count, (ii) fewer barrier function constraints per robot in the next stage, reducing computation and control effort, and (iii) robustness to drift via short alternative paths and triangle backups.

B. Decentralized CLF-CBF Control on the Evolving Tree

Once pruning yields the sparse backbone $\mathcal{G}' = (\mathcal{V}, \mathcal{E}')$, each robot i maintains two neighbor sets: the *critical neighbors* $\mathcal{C}_i = \{j : (i, j) \in \mathcal{E}'\}$, which belong to the pruned structure and must be preserved for connectivity, and the *sensing neighbors* $\mathcal{N}(i) = \{j : \|x_i - x_j\| \leq R_{\text{max}}\}$, which are considered for collision safety.

For every $j \in \mathcal{N}(i)$, a safety barrier candidate

$$h_{ij}^{\text{safe}} = \|x_i - x_j\|^2 - d_{\min}^2$$

enforces collision avoidance via establishing the inequality, and for every $j \in \mathcal{C}_i$, a connectivity barrier candidate

$$h_{ij}^{\text{conn}} = R_{\text{max}}^2 - \|x_i - x_j\|^2$$

works in parallel with the safety barrier candidate to ensure that critical links remain within range. These constraints are standard in multi-robot control and are implemented in real time as introduced in [17], [25].

The Lyapunov function is defined over the candidate

$$V_i(x_i) = \|x_i - x_i^{\text{ref}}\|^2,$$

where the reference x_i^{ref} depends on whether a target is detected. If no target is detected, $x_i^{\text{ref}} = x_i$ and $V_i \equiv 0$, therefore the inequality upon this candidate is vacuous. If a target is assigned, $x_i^{\text{ref}} = x_i^{\text{goal}}$, and the inequality drives x_i toward the target.

Algorithm 1 Distributed Distance-First Pruning with Recouping and Local Rewiring

- 1: **Input:** Graph $G = (V, E)$, neighbor sets $\mathcal{N}(i)$, critical edges $\mathcal{C}$ from Sec. III-A
- 2: **Output:** Reduced/updated edge set E'
- 3: $E' \leftarrow E$; $E^- \leftarrow E' \setminus \mathcal{C}$ {Only non-critical edges can be pruned}
- 4: Each endpoint of $(i, j) \in E^-$ measures $l_{ij} = \|x_i - x_j\|$ and exchanges l_{ij}
- 5: Rank edges in E^- by descending l_{ij} ; within a small length tolerance, use P_{bil} as an optional robustness tie-breaker (higher P_{bil} is more valuable to keep)
- 6: Build a vertex-disjoint batch $\mathcal{E}_{\text{batch}}$ from the ranked list
- 7: **for each** $(i, j) \in \mathcal{E}_{\text{batch}}$ **do**
- 8: Check alternative path $i \rightarrow \dots \rightarrow j$ within H hops whose maximum edge length $< l_{ij}$
- 9: Degree guard: $\min\{\deg_{E'}(i), \deg_{E'}(j)\} \geq 2$
- 10: Next-step guard: all edges on the alternative path satisfy $l \leq R_{\text{max}} - \varepsilon$
- 11: **if** all guards satisfied **then**
- 12: $E' \leftarrow E' \setminus \{(i, j)\}$ {Prune}
- 13: **end if**
- 14: **end for**
- 15: **Recoup:** If any node's degree becomes 0, restore its most recently removed incident edge
- 16: **for each** $i \in V$ **do**
- 17: **for each** $k \in \mathcal{N}^{(2)}(i)$ with $(i, k) \notin E'$ **do**
- 18: {2-hop candidates} $S_{\text{distance}} = \frac{R_{\text{max}} - \|x_i - x_k\|}{R_{\text{max}}}$;
- 19: $S_{\text{connectivity}} = 1 + \frac{|\mathcal{CN}(i, k)|}{2}$; $S_{\text{efficiency}} = \frac{1}{d_{G'}(i, k)}$
- 20: $S_{\text{add}}(i, k) = \alpha S_{\text{distance}} + \beta S_{\text{connectivity}} + \gamma S_{\text{efficiency}}$
- 21: **end for**
- 22: Propose addition (i, k^*) with maximal S_{add} that pre-serves connectivity and reduces local max incident length
- 23: **if** proposal accepted by endpoints **then**
- 24: $E' \leftarrow E' \cup \{(i, k^*)\}$ {Local rewiring}
- 25: **end if**
- 26: **end for**
- 27: **return** E'

Scenario 1: Exploration (no target). All robots drift under the background flow f_{flow} . Only the CBFs are active: safety barriers prevent collisions with sensing neighbors, and connectivity barriers preserve links with critical neighbors. Since $V_i \equiv 0$ and the inequality established by default, no additional control action is applied, the fleet explores passively while maneuver itself only when the sparse links in the pruned critical structure is at risk.

Scenario 2: Target engagement. When a robot detects a target, V_i turns to a positive value with $x_i^{\text{ref}} = x_i^{\text{goal}}$. Solving control inputs satisfying the inequality to bring down V_i drives this detecting robot toward the goal. The same set of connectivity barriers now drives the rest robots to stay in range with their critical neighbors, yielding a connected structure being ‘dragged’ by the detecting robot towards the

Algorithm 2 End-to-End Decentralized Control Framework (per timestep)

- 1: Neighbor exchange; update $\mathcal{N}_i$ and run distributed critical-edge detection [24]
 - 2: Distance-first pruning on $E \setminus \mathcal{C}$ with guards and batching; optional triangle tie-breaker; recoup/rewire locally (Alg. 1)
 - 3: Set $\mathcal{T}_i \leftarrow \{j : (i, j) \in E'\}$ from current pruned graph
 - 4: **if** agent i has a target in range **then**
 - 5: Activate CLF toward x_i^{goal} ; solve CLF-CBF constrained control problem with safety CBFs for $\mathcal{N}_i$ and connectivity CBFs for $\mathcal{T}_i$
 - 6: **else**
 - 7: Solve CBF-only problem (safety on $\mathcal{N}_i$, connectivity on $\mathcal{T}_i$); drift under flow otherwise
 - 8: **end if**
 - 9: Apply u_i ; broadcast updated state to neighbors
-

emerging target.

The overall control framework is summarized as in Alg. 2.

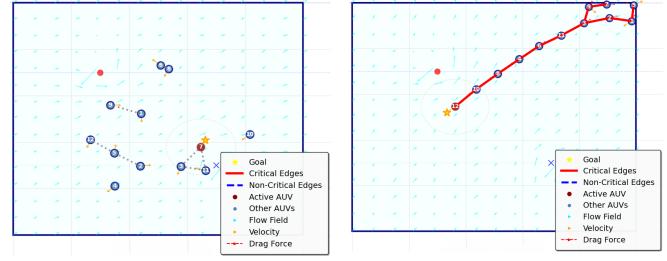
IV. SIMULATION SETUP AND RESULTS

In this section we present several simulated examples to demonstrate the results of the proposed framework. In each run, N robots (fixed $N = 12$ or randomized $N \in [3, 20]$) operate in a 10×10 m² workspace with discrete-time integration ($\Delta t = 0.1$ s). Robots have randomized hydrodynamic parameters: mass $m \approx 2.0 \pm 0.2$ kg, volume $V = m/1800$ m³, drag coefficient $C_d \approx 0.8 \pm 0.1$, and frontal area $A \approx 0.01 \pm 0.002$ m². Speeds are capped at 2.0 m/s. Communication uses a nominal range $R_{\text{max}} = 1.2$ m, attenuated by spreading loss ($20 \log_{10} r$), absorption (0.1 dB/m), and depth-layer factors, with a guaranteed minimum of $0.3R_{\text{max}}$. A safety distance $d_{\text{min}} = 0.6$ m is enforced.

Target engagement is handled through a decentralized CLF-CBF quadratic program (QP) with parameters $\alpha = 1.5$, $\beta = 5.0$, $\beta_s = 10.0$, input bound $\|u\| \leq 0.40$, and cost $\|u\|^2 + 50\gamma^2$. The QP is solved online via CVXPY (SciPy fallback). Exploring robots apply a CBF-enhanced force-based controller with repulsive safety and connectivity terms.

The environment combines a rotating base current $\mathbf{V}(t)$ (initialized at $[0.2, 0.1]$ m/s), two Gaussian vortices at $(3, 7)$ and $(7, 3)$ with strengths $+1.5$ and -1.2 and radii 2.0, 1.8, and sinusoidal turbulence of intensity 0.05. Hydrodynamic drag is modeled with buoyancy and advection forces included for realism.

Figure 2 provides a direct comparison between the uncontrolled and controlled cases. In both panels, the same set of robots experiences identical environmental dynamics and parameters. In the uncontrolled case (Fig. 2a), robots drift apart, their communication links vanish, and the network fragments into several disconnected clusters. In contrast, with our decentralized control framework (Fig. 2b), connectivity is preserved throughout the drift and the team remains coordinated while progressing toward the goal.



(a) Brownian motion: edges vanish, network disconnects.

(b) Proposed method: connectivity preserved.

Fig. 2. Comparison between uncontrolled drift and our decentralized framework. In the uncontrolled case, robots diffuse apart and the network fragments, whereas our method preserves connectivity and still reaches the goal within a comparable time.

It is generally expected that in the absence of globally shared information, and under continuously changing environments, distributed methods will lose some optimality compared to centralized solutions. To evaluate this tradeoff, we conducted a simulation in which our distributed pruning method (Sec. III.A, without the optional robustness enhancements) was run in parallel with a centralized minimum spanning tree (MST) identification. The resulting connectivity graphs were compared over time.

Figure 3 shows that by the end of the mission the distributed strategy produces a graph with only 11 active edges for 12 robots, matching the edge count of the MST. This outcome is quantified in Fig. 4, which plots the total edge length of the distributed method against the centralized MST. The green curve (proposed framework) closely tracks the blue curve (centralized MST), and the red curve (the difference between the two) steadily decreases to zero as the robots disperse. The small bumps on the red curve correspond to times when new targets are detected and the team reconfigures, temporarily disturbing the topology. Despite these events, the distributed strategy consistently tracks the MST and converges to the same total cost, demonstrating the efficiency of local pruning decisions in the long run.

While the previous analysis compared only the graph structures generated by our decentralized framework and a centralized MST, Figure 5 evaluates the full closed-loop performance when each structure is used as the backbone of a CLF-CBF controller. In this setting, the MST-based CLF-CBF serves as a close approximation of the approach in [17], whereas our framework applies the controller on top of a *distributedly generated* critical structure. The side-by-side results show that both strategies successfully guide the robots to the goal while preserving connectivity, with our method achieving performance comparable to the centralized baseline despite foregoing centralized spanning graph construction.

See full video¹ for all simulated examples presented in this section.

¹https://youtu.be/FbIUaF_sbOM

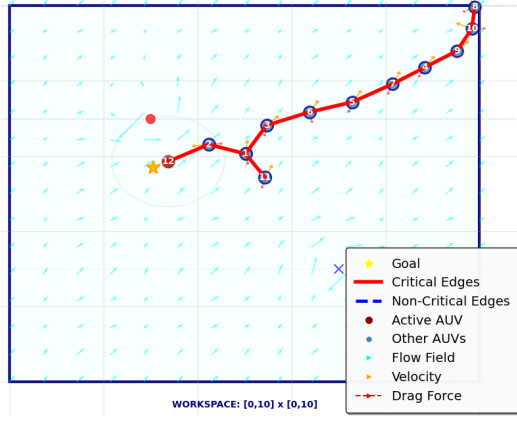


Fig. 3. 12-robot in the target engagement scenario. The decentralized framework achieves 11 critical edges (solid lines) in the final state, preserving connectivity while minimizing total distance.

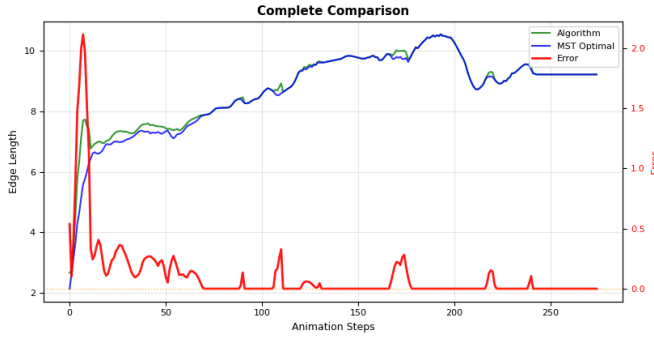


Fig. 4. The decentralized framework prunes edges and produces the preserved critical structure (green) closely matches the MST (blue) in total edge lengths. Over time when robots disperse, the error metric (red) falls to zero, confirming that local pruning recovers an MST-equivalent structure in the long run.

V. CONCLUSION AND FUTURE WORK

We presented a distributed framework for underwater robot fleets operating in flow-dominated environments where localization, communication, and energy resources are limited. Our approach combines: (i) distributed identification of a sparse critical structure, and (ii) decentralized CLF-CBF control layered on this structure to maintain safety and connectivity during drift and to reconfigure reliably when targets emerge. Through this design, robots exploit natural flow for energy-efficient exploration while preserving end-to-end connectivity and responding adaptively to new goals.

The results demonstrate that connectivity can be maintained without global knowledge on robot locations, and that the fleet remains robust to emerging targets and environmental disturbances.

Future work will first focus on incorporating realistic underwater vehicle dynamics into the framework and validating the approach with physical robot experiments. Beyond this natural extension, we will explore distributed methods for synthesizing alternative network topologies tailored to different tasks, such as cooperative localization, rendezvous of robots, and persistent environmental monitoring. By adapting

the underlying connectivity structure to the requirements of specific missions, we aim to extend the flexibility and applicability of the proposed framework.

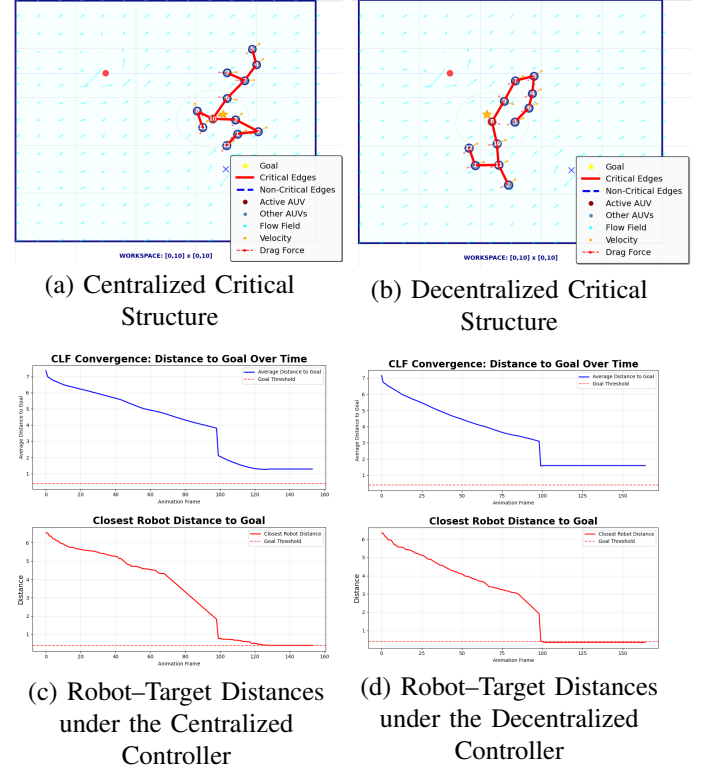


Fig. 5. Comparison of centralized and decentralized critical structure identification in a 12-robot target engagement scenario under identical underwater settings. (a) Final state robot network with centralized identified MST. (b) Final state robot network with decentralized identified critical structure. (c) Average and minimum robot-target distances under the centralized controller. (d) Average and minimum robot-target distances under the decentralized controller. Both methods preserve connectivity and drive the team to the goal with comparable performance.

REFERENCES

- [1] A. Muto, S. Athar, N. Butler, J. Garwood, C. Gibson, G. Hollinger, P. Lundrigan, J. Montierth, N. Rypkema, B. Schiel *et al.*, “Meshed observations of the remote subsurface with heterogeneous intelligent platforms (mothership): development of mothership-and-passenger sampling system for ice-sheet grounding-zone observations: Early results,” in *AGU Fall Meeting Abstracts*, vol. 2024, no. 466, 2024, pp. C33D–0466.
- [2] B. Schiel, J. Montierth, E. Blattner, C. E. Dobbs, T. B. Nielsen, and P. Lundrigan, “Using orthogonal chirps underwater for in-band, full-duplex communication with minimal self-interference cancellation,” in *Proceedings of the 18th International Conference on Underwater Networks & Systems*, 2024, pp. 1–8.
- [3] B. Schiel and P. Lundrigan, “Role-based network addressing for fleets of autonomous underwater vehicles,” in *Proceedings of the 18th International Conference on Underwater Networks & Systems*, 2024, pp. 1–2.
- [4] N. L. Butler and G. A. Hollinger, “Hybrid decentralization for multi-robot orienteering with mothership-passenger systems,” in *2025 IEEE American Conference on Robotics and Automation (ICRA)*. IEEE, 2025, pp. 6573–6579.
- [5] C. Wei, H. G. Tanner, X. Yu, and M. A. Hsieh, “Low-range interaction periodic rendezvous along lagrangian coherent structures,” in *2019 American control conference (ACC)*. IEEE, 2019, pp. 4012–4017.
- [6] G. Knizhnik, P. Li, X. Yu, and M. A. Hsieh, “Flow-based control of marine robots in gyre-like environments,” in *2022 International Conference on Robotics and Automation (ICRA)*. IEEE, 2022, pp. 3047–3053.

- [7] G. I. Taylor, "Diffusion by continuous movements," *Proceedings of the london mathematical society*, vol. 2, no. 1, pp. 196–212, 1922.
- [8] A. Okubo, "Oceanic diffusion diagrams," in *Deep sea research and oceanographic abstracts*, vol. 18, no. 8. Elsevier, 1971, pp. 789–802.
- [9] A. J. Majda and P. R. Kramer, "Simplified models for turbulent diffusion: theory, numerical modelling, and physical phenomena," *Physics reports*, vol. 314, no. 4-5, pp. 237–574, 1999.
- [10] F. Berlinger, M. Gauci, and R. Nagpal, "Implicit coordination for 3d underwater collective behaviors in a fish-inspired robot swarm," *Science Robotics*, vol. 6, no. 50, p. eabd8668, 2021.
- [11] A. Quattrini Li, C. J. Carver, Q. Shao, X. Zhou, and S. Nelakuditi, "Communication for underwater robots: Recent trends," *Current Robotics Reports*, vol. 4, no. 2, pp. 13–22, 2023.
- [12] R. Olfati-Saber, J. A. Fax, and R. M. Murray, "Consensus and cooperation in networked multi-agent systems," *Proceedings of the IEEE*, vol. 95, no. 1, pp. 215–233, 2007.
- [13] D. Di Martino, A. Germani, C. Manes, and P. Palumbo, "Design of observers for systems with rational output function," in *Proceedings of the 45th IEEE Conference on Decision and Control*, 2006, pp. 1641–1646.
- [14] B. Capelli and L. Sabattini, "Connectivity maintenance: Global and optimized approach through control barrier functions," in *2020 IEEE International Conference on Robotics and Automation (ICRA)*, 2020, pp. 5590–5596.
- [15] L. Sabattini, N. Chopra, and C. Secchi, "Decentralized connectivity maintenance for cooperative control of mobile robotic systems," *The International Journal of Robotics Research*, vol. 32, no. 12, pp. 1411–1423, 2013.
- [16] B. Capelli, H. Fouad, G. Beltrame, and L. Sabattini, "Decentralized connectivity maintenance with time delays using control barrier functions," in *2021 IEEE International Conference on Robotics and Automation (ICRA)*, 2021, pp. 1586–1592.
- [17] Y. Yang, Y. Lyu, and W. Luo, "Minimally constrained multi-robot coordination with line-of-sight connectivity maintenance," *arXiv preprint arXiv:2303.04271*, 2023.
- [18] O. Borvka, "O jistém problému minimálním," 1926.
- [19] J. B. Kruskal, "On the shortest spanning subtree of a graph and the traveling salesman problem," *Proceedings of the American Mathematical society*, vol. 7, no. 1, pp. 48–50, 1956.
- [20] R. C. Prim, "Shortest connection networks and some generalizations," *The Bell System Technical Journal*, vol. 36, no. 6, pp. 1389–1401, 1957.
- [21] R. G. Gallager, P. A. Humblet, and P. M. Spira, "A distributed algorithm for minimum-weight spanning trees," *ACM Transactions on Programming Languages and systems (TOPLAS)*, vol. 5, no. 1, pp. 66–77, 1983.
- [22] B. Awerbuch, "Optimal distributed algorithms for minimum weight spanning tree, counting, leader election, and related problems," in *Proceedings of the nineteenth annual ACM symposium on Theory of computing*, 1987, pp. 230–240.
- [23] M. Khan and G. Pandurangan, "A fast distributed approximation algorithm for minimum spanning trees," *Distributed Computing*, vol. 20, no. 6, pp. 391–402, 2008.
- [24] D. B. Venkateswaran, Z. Qu, and A. Gusrialdi, "A distributed method for detecting critical edges and increasing edge connectivity in undirected networks," in *2024 IEEE 63rd Conference on Decision and Control (CDC)*, 2024, pp. 6951–6956.
- [25] A. D. Ames, S. Coogan, M. Egerstedt, G. Notomista, K. Sreenath, and P. Tabuada, "Control barrier functions: Theory and applications," in *2019 18th European Control Conference (ECC)*, 2019, pp. 3420–3431.